

NexaFlow

Predicción de demanda

Maqueta del informe de la EPE 3

Teoría y Aplicaciones de la Inteligencia Artificial

IPCHILE

Estudiante: [Completar]

Carrera y sección: [Completar]

Docente: [Completar]

Fecha de entrega: [Completar]

Versión de trabajo con evidencia técnica local

Datos sintéticos

Validación empresarial y usabilidad pendientes

Cómo usar esta maqueta

Desarrolla los apartados marcados Completar, conserva la trazabilidad de los resultados y reemplaza las cifras si vuelves a entrenar. Esta versión organiza el trabajo y no constituye un informe final listo para entregar.

Introducción

Este documento organiza el desarrollo de la EPE 3 mediante una aplicación de predicción de demanda para el caso NexaFlow. La propuesta transforma las variables operativas de una zona en una estimación diaria de unidades y permite consultar el modelo desde una interfaz web conectada a una API.

La versión actual es una maqueta de trabajo con un prototipo ejecutable. Los resultados técnicos provienen de una ejecución sobre datos sintéticos generados para el ejercicio. No se dispone de registros empresariales, de la EPE 2 completa ni de pruebas con usuarios. Por eso, las métricas no acreditan mejoras reales de inventario, ahorro ni capacidad productiva.

La pauta solicita informe en PDF, código en GitHub y presentación PowerPoint con evidencia. El informe utiliza Calibri 12, márgenes de 3 cm, interlineado 1,5 y alineación justificada. Se reservan diez páginas para el desarrollo, organizadas en las cuatro partes de la actividad. Los espacios de trabajo deben convertirse en análisis y evidencias antes de la entrega final.

El alcance implementado comprende demanda agregada por zona, comparación de modelos, servicio FastAPI, formulario Flask, pruebas automatizadas y carga HTTP local. La evaluación empresarial y la usabilidad con participantes se deben completar por separado.

1 Diseño del problema empresarial

Parte 1 de la pauta. NexaFlow se utiliza como caso de continuidad sugerido por el estudiante. La necesidad propuesta es anticipar la demanda diaria por zona para apoyar la revisión de inventario. Debe verificarse que esta necesidad coincide con el diagnóstico y los antecedentes de la evaluación anterior.

La unidad de análisis implementada es una zona en un día. El objetivo es estimar unidades agregadas; no existe una variable de producto ni una optimización de rutas. Para desagregar por producto se necesitarían su identificador, historial suficiente y un rediseño de la preparación y evaluación de datos.

El usuario previsto es una persona responsable de planificación o reposición. Ingresar zona, día, mes, promoción y demanda de siete días antes. La aplicación presenta la estimación y la advertencia de datos sintéticos. La decisión final requiere revisar existencias, pedidos pendientes, capacidad y restricciones operativas.

El beneficio esperado es reducir incertidumbre al planificar. Esta expectativa debe comprobarse mediante un piloto que compare decisiones y costos con y sin apoyo del modelo. No se puede deducir ahorro financiero únicamente desde MAE o R^2 .

Completar: describir empresa y área, problema observado, evidencia del diagnóstico anterior, quién toma la decisión y qué consecuencia tiene equivocarse. Incorporar un indicador empresarial con fuente y período. No inventar montos, porcentajes ni entrevistas.

2 Objetivos y requisitos de la solución

Objetivo general propuesto: desarrollar y evaluar un prototipo web que estime demanda diaria por zona y comunique el resultado mediante una API. Los objetivos específicos son construir un conjunto reproducible, comparar modelos con separación temporal y comprobar el funcionamiento del flujo de consulta.

Requisitos funcionales implementados: ingresar cinco variables, solicitar una predicción, mostrar unidades, rechazar entradas fuera de rango y comunicar indisponibilidad del servicio. La API expone POST /predict y GET /health; la documentación interactiva se encuentra en /docs.

Requisitos de calidad propuestos: conservar entradas cuando ocurre un error, mostrar etiquetas comprensibles, responder sin exponer detalles internos y mantener resultados reproducibles. La carga local sirve como observación inicial de tiempos, no como garantía de servicio.

Criterio de aceptación del modelo: comparar el error con una línea base de persistencia semanal y evaluar si la magnitud resulta útil para el negocio. La reducción de error necesaria debe acordarse con responsables operativos, considerando costos de sobrestock y desabastecimiento.

Completar: definir usuarios, volumen de consultas esperado, objetivo de latencia y umbral de error aceptable. Marcar cada requisito como implementado, verificado o pendiente y vincularlo con una evidencia concreta.

3 Datos y preparación temporal

El generador produce 2896 filas útiles de cuatro zonas entre 2023 y 2024, con semilla 42. Simula una demanda aditiva influida por zona, fin de semana, promoción, estacionalidad mensual y ruido. Esta formulación facilita una demostración reproducible y puede favorecer la regresión lineal.

Las variables son zona (1 a 4), día de semana (0 a 6), mes (1 a 12), promoción (0 o 1) y demanda_lag7. El objetivo demanda es una cantidad continua de unidades. La fecha determina la separación temporal y no entra directamente en el estimador.

El rezago se construye dentro de cada zona con el valor observado siete días antes. Se eliminan los primeros siete registros de cada zona. El calendario sintético es completo; con registros reales habría que distinguir días sin actividad de datos faltantes y reindexar antes de calcular el rezago.

La codificación one-hot de zona, día y mes se encuentra dentro del pipeline. Cada entrenamiento aprende sus transformaciones utilizando solo su partición de entrenamiento. El conjunto de test no interviene en la selección del modelo.

Completar: diccionario empresarial, origen y autorización de datos, fecha de extracción, cobertura, duplicados, faltantes y tratamiento de ventas limitadas por falta de stock. Las ventas observadas pueden subestimar demanda no atendida.

4 Arquitectura y flujo de predicción

La arquitectura separa la presentación de la inferencia. El navegador envía el formulario a Flask en el puerto 5000. Flask transforma los campos numéricos y realiza una solicitud HTTP a FastAPI en el puerto 8000. FastAPI valida los valores y consulta el modelo cargado en memoria.

El entrenamiento ocurre fuera de las solicitudes web. `train_model.py` guarda el pipeline, nombres de variables y nombre del modelo en `artifacts/modelo.pkl`. La API carga ese archivo al iniciar; si no existe, devuelve un estado de indisponibilidad con una instrucción de recuperación.

El resultado incluye `prediccion_demanda`, unidad, modelo y origen sintético de los datos. Flask muestra la cantidad y mantiene los campos seleccionados. Un timeout evita que la web espere indefinidamente cuando falla la comunicación.

Los archivos se ubican con rutas relativas al código cuando corresponde. El endpoint del servicio puede configurarse con `NEXAFLOW_API_URL`. El prototipo se ejecuta en la interfaz local del equipo y requiere iniciar ambos procesos.

Completar: incorporar un diagrama propio del flujo navegador, Flask, FastAPI y modelo. Explicar los puertos, la respuesta ante fallos y por qué la separación permite cambiar la interfaz sin volver a entrenar.

5 Entrenamiento y selección del modelo

Parte 2 de la pauta. Se comparan una regresión lineal y cuatro configuraciones de Random Forest con 100 árboles, profundidad máxima de 6 o 12 y mínimo de 2 o 5 muestras por hoja. Esta búsqueda acotada permite mostrar ajuste de hiperparámetros sin afirmar que se exploraron todas las alternativas.

El primer 80% de las fechas aporta 2316 registros de desarrollo. Sobre estas fechas se ejecuta TimeSeriesSplit con cinco ventanas crecientes. Las cuatro zonas de una fecha siempre pertenecen a la misma partición. El test final contiene 580 registros desde 2024-08-09.

La selección utiliza el menor MAE medio de validación. Después se ajusta el candidato seleccionado con todo el bloque de desarrollo y se evalúa una vez en el test reservado. Se conserva ese modelo evaluado para servir predicciones, sin reentrenarlo con el test.

El candidato seleccionado fue Lineal, con MAE medio de validación 9.26. Los resultados detallados por ventana y los límites de fechas están en metricas.json. La comparación no demuestra que ese algoritmo será superior con demanda real.

Completar: insertar tabla con los cinco candidatos, comentar estabilidad entre ventanas y relacionar la elección con el análisis previo. Explicar que el rezago del test usa observaciones pasadas disponibles para cada predicción diaria, no un pronóstico multihorizonte.

6 Interfaz y contrato de la API

La interfaz utiliza listas de selección para zona, día, mes y promoción, y un campo numérico para demanda histórica. Las etiquetas están asociadas a los controles, el resultado se anuncia en una región de actualización y el diseño reorganiza las columnas en pantallas pequeñas.

El contrato recibe enteros estrictos para zona, día, mes y promoción. La demanda debe ser numérica, finita y no negativa, con un límite técnico de 100000. Se rechazan campos adicionales. El límite requiere revisión al incorporar el contexto empresarial.

Una solicitud válida responde con código 200. FastAPI devuelve 422 ante un cuerpo inválido y 503 si el modelo no está cargado. La web presenta mensajes comprensibles y devuelve 400 por datos incorrectos o 503 por indisponibilidad. La documentación /docs permite probar el contrato.

En la ejecución comprobada, zona 1, sábado, diciembre, promoción activa y demanda_lag7 de 95.0 produjeron 127.8 unidades. El request y response completos se guardan en ejemplo_api.json. Es una predicción de demostración sobre el modelo sintético.

Completar: incorporar capturas del formulario, del resultado y de una entrada inválida. Describir la diferencia entre validar el formulario y validar la API, dado que un cliente externo puede omitir la interfaz.

7 Métricas y análisis de errores

Parte 3 de la pauta. Se utilizan MAE y RMSE porque el objetivo es continuo. MAE mide el error absoluto promedio en unidades; RMSE da mayor peso a errores grandes. R^2 es una medida complementaria de variación explicada. Precision, Recall y F1 corresponden a clasificación y no se aplican directamente a esta regresión.

En el test reservado, el modelo obtuvo MAE 6.41 unidades, RMSE 8.07 unidades y R^2 0.842. La persistencia semanal, que repite demanda_lag7, obtuvo MAE 13.70 y RMSE 17.51. Los valores se calcularon sobre las mismas filas.

El menor error del modelo frente a esta línea base respalda su utilidad para la demostración. No prueba reducción de quiebres de stock ni ahorros reales. Tampoco significa que todas las predicciones estarán dentro de un margen igual al MAE.

El archivo predicciones_test.csv permite revisar errores por zona, promociones y fecha. Ese análisis debe acompañar las métricas globales para detectar segmentos donde una buena media oculte un comportamiento deficiente.

Completar: agregar tabla de error por zona y gráfico de observado frente a predicho. Analizar al menos dos errores grandes y su posible consecuencia operativa. No volver a seleccionar hiperparámetros usando el test sin reservar una nueva evaluación independiente.

8 Pruebas funcionales y rendimiento

La suite automatizada terminó con 16 pruebas aprobadas. Incluye predicción válida, validación de rangos y tipos, claves adicionales, cuerpos incompletos, modelo ausente, carga del formulario, comunicación web/API con cliente de pruebas, mensaje ante desconexión y límites temporales.

La integración de prueba utiliza TestClient para FastAPI y el cliente de Flask. Además se ejecutó el flujo real por HTTP y se capturó su resultado desde un navegador. Las pruebas no equivalen a una auditoría de seguridad ni a una evaluación con usuarios.

La medición local envió 100 solicitudes con concurrencia de 10 a cada flujo. API: 0 errores y p95 52.2 ms. Web con consulta a API: 0 errores y p95 96.3 ms. Se hizo una solicitud previa de calentamiento por flujo.

Los resultados corresponden a una ejecución breve en Windows con servicios locales y sin tráfico externo. No describen capacidad sostenida, latencia de internet ni tolerancia a fallos bajo condiciones productivas. El script incluye timeout y registra solicitudes fallidas.

Completar: repetir benchmark en el equipo de presentación, registrar hardware, versiones y fecha. Aumentar escenarios de carga según el uso esperado y explicar qué se optimizó tras observar los resultados.

9 Usabilidad y riesgos operativos

La inspección del navegador verificó que el formulario permite solicitar una predicción y que la página móvil no presenta desbordamiento horizontal. Esto es una comprobación técnica de interfaz. No se realizaron sesiones con participantes y no se dispone de resultados de usabilidad humana.

Protocolo propuesto: invitar de tres a cinco personas representativas a generar una predicción, cambiar la promoción, recuperarse de un dato inválido y explicar el resultado. Registrar éxito por tarea, tiempo, errores y claridad percibida, sin datos personales innecesarios.

Se proponen como metas iniciales al menos 80% de tareas completas y mediana inferior a 60 segundos. Estas metas deben acordarse antes de las sesiones; no son resultados obtenidos. PLAN_DE_TRABAJO.md contiene una tabla para registrar las observaciones.

Riesgos: datos sintéticos poco representativos, cambio de patrón de demanda, rezagos incorrectos y uso del resultado como orden automática. El prototipo no incluye autenticación ni despliegue productivo. Solo carga un modelo generado localmente y necesita validación adicional antes de uso operacional.

Completar: aplicar el protocolo, presentar resultados anónimos y describir al menos una mejora basada en observación. Definir quién revisará deriva de datos, frecuencia de reentrenamiento y condiciones para retirar un modelo.

10 Documentación y presentación de resultados

Parte 4 de la pauta. El paquete contiene código, dependencias fijadas, conjunto sintético, modelo entrenado, métricas, pruebas, carga y capturas. README.md explica cómo crear el entorno, entrenar, iniciar los servicios y consultar la API. El modelo conserva el hash registrado en metrics.json para identificar el archivo evaluado.

La presentación ejecutiva resume problema, solución, método, resultados medidos y límites. Incluye una captura real de la aplicación y cifras de la ejecución local. Antes de entregar, debe incorporar nombre del estudiante, identificación del curso y enlace al repositorio publicado.

El repositorio está preparado localmente, pero no está publicado en GitHub. La entrega requiere crear el repositorio en la cuenta del estudiante, subir el código y verificar las instrucciones desde un clon limpio. No sustituir este paso por un enlace ficticio.

Guion sugerido: explicar la decisión empresarial, demostrar una predicción, describir la separación temporal, presentar MAE/RMSE y mencionar las limitaciones. Cerrar con los pasos necesarios para validar datos y utilidad empresarial. Evitar afirmar que el sistema está listo para producción.

Completar: URL real del repositorio, inventario de evidencias, instrucciones de reproducción verificadas y captura de la versión entregada. Revisar que el desarrollo final tenga de diez a quince páginas sustantivas, sin contar portada, introducción, conclusión ni bibliografía.

Conclusión

El prototipo demuestra una integración funcional entre una interfaz Flask, una API FastAPI y un modelo de Scikit-learn. La separación temporal, las pruebas automatizadas y la medición HTTP ofrecen evidencia reproducible del funcionamiento técnico en el entorno local.

La regresión lineal seleccionada alcanzó MAE 6.41 y RMSE 8.07 en el test sintético. Superó a la persistencia semanal en ese experimento. La estructura aditiva del generador y la ausencia de datos reales limitan cualquier extrapolación a la empresa.

Para completar la actividad se debe vincular el proyecto con la evaluación anterior, incorporar antecedentes empresariales verificables, realizar las pruebas de usabilidad y publicar el código. La validación con demanda real permitirá discutir si los errores son aceptables para la decisión de inventario.

Completar: redactar una reflexión propia sobre lo aprendido, decisiones de diseño, dificultades y mejoras. Distinguir resultados alcanzados, expectativas y trabajo futuro.

Bibliografía

IPCHILE. (2024). Diseño, implementación y evaluación de una aplicación web con Machine Learning en un entorno empresarial real [Pauta de actividad evaluativa]. Material proporcionado por el estudiante.

Scikit-learn developers. (s. f.). Cross-validation: Evaluating estimator performance. Scikit-learn. Recuperado el 14 de septiembre de 2026, de https://scikit-learn.org/stable/modules/cross_validation.html

FastAPI. (s. f.). Request body. Recuperado el 14 de septiembre de 2026, de <https://fastapi.tiangolo.com/tutorial/body/>

Pallets. (s. f.). Quickstart. Flask documentation. Recuperado el 14 de septiembre de 2026, de <https://flask.palletsprojects.com/en/stable/quickstart/>

Completar: agregar la referencia exacta de la EPE 2 y del material docente que efectivamente se utilice. Verificar autor institucional y año de la pauta contra la información del curso. No citar apuntes no revisados como respaldo de conclusiones.